



Fiti – Smart Tailoring & Custom Fashion Platform

Module: CCS2313 - Project Management

Date: July 13, 2026

- Fathima Zainab Ilyas (CIT-24-02-0182)
- Oshanda Induwara (CIT-24-02-0125)
- Lamindu Rathnayake (CIT-24-02-0030)
- Ahmed Rafnas (CIT-24-02-0064)
- Sakila Induwara (CIT-24-02-0113)

Table of Contents

1	PROJECT TITLE & DESCRIPTION	
1.1	PROJECT TITLE	2
1.2	BACKGROUND & PROBLEM JUSTIFICATION	2
1.3	EXPECTED IMPACT & MARKET GAP	2
1.4	SYSTEM ARCHITECTURE & WORKFLOW OVERVIEW	3
2	OBJECTIVES & SCOPE	
2.1	OBJECTIVES	3
2.2	SCOPE OF THE SYSTEM	3
3	INITIAL TIMELINE (GANTT CHART)	
3.1	MILESTONE SCHEDULE AND STATUS MATRIX	5
3.2	RISK MITIGATION & CONTINGENCY STRATEGY	6
4	TEAM ROLES & RESPONSIBILITIES	
4.1	GROUP PERSONNEL ALLOCATIONS	6
4.2	EQUIPMENT, INFRASTRUCTURE & RESOURCES	7
4.3	FINANCIAL FEASIBILITY & BUDGET SUMMARY	7

1 Project Title & Description

1.1 Project Title

Fiti – Smart Tailoring & Custom Fashion Platform

1.2 Background & Problem Justification

The digital transformation of service-based industries has significantly improved accessibility, efficiency, and user experience. However, certain sectors, such as tailoring services, continue to rely heavily on traditional, manual processes. These processes often involve face-to-face communication, verbal instructions, and informal pricing negotiations, which can lead to inefficiencies, miscommunication, and limited customer reach. In many cases, customers struggle to clearly explain their design requirements, while tailors face challenges in managing multiple orders and attracting new clients. Additionally, there is no standardized mechanism to compare pricing, delivery timelines, or service quality among different tailors.

1.3 Expected Impact & Market Gap

Although several platforms such as Indochino and Sumissura provide custom tailoring services, they primarily focus on standardized online ordering and lack interactive features such as real-time bidding and personalized communication. Similarly, service marketplaces like Fiverr and TaskRabbit support service requests and provider matching but are not specialized for tailoring workflows.

This highlights a clear market gap where no unified platform exists to connect customers and tailors through design-based requests, voice communication, transparent bidding, and location-based matching. The proposed Fiti system aims to bridge this gap by integrating all these features into a single, user-friendly platform.

1.4 System Architecture & Workflow Overview

Fiti is a web-based platform designed to digitally connect customers, tailors, and fashion designers to simplify the process of custom clothing creation. The system begins with user authentication (including Google login), after which customers can view nearby tailors using auto location detection. Customers can either upload design inspirations (images/screenshots from platforms like Pinterest) or request assistance from fashion designers if they lack ideas.

Once a request is submitted, tailors can review it and place bids based on pricing and feasibility. Customers can compare bids based on price, ratings, and proximity, and select a tailor accordingly. Real-time notifications keep both parties updated throughout the process. The platform also includes separate dashboards for customers and tailors, enabling efficient order management. Customers can choose between physical visits or fully online interactions, including submitting measurements digitally.

The system integrates a mock payment mechanism to simulate transaction flow for the MVP. Additionally, shop management is treated as an entity within the system, ensuring that tailoring businesses are properly represented.

2 Objectives & Scope

2.1 Objectives

- To develop a centralized digital platform connecting customers and tailors.
- To simplify custom clothing ordering using image-based design sharing.
- To enable price comparison through a competitive marketplace bidding system.
- To provide automated, location-based tailor shop discovery.
- To support flexible, hybrid tailoring workflows (both online and physical shop visits).
- To improve clarity of communication between customers and tailors through digital channels.
- To simulate an end-to-end payment confirmation transaction flow within the MVP scope.
- To provide an intuitive user experience utilizing highly distinct role-based dashboards.

2.2 Scope of the System

In-Scope Features (Core Functionalities)

- User registration & login (email/password + Google login).
- Role-based access validation (Customer / Tailor interface views).
- Auto location detection for nearby tailor discovery.
- Tailor shop as an entity (shop profile creation & management).

- Design upload interfaces supporting images and screenshots.
- Voice note instructions upload for custom customer order specifications.
- Order request creation pipeline.
- Tailor marketplace interaction engine (price-based competitive bidding offers).
- Bid evaluation overview (viewing price, shop ratings, and location proximity).
- Customer acceptance action workflow for selecting a preferred tailor.
- Real-time notifications triggered by milestone updates.
- Separate functional dashboards for customers and tailors.
- Measurement submission framework built for manual numerical text inputs.
- Flexible service operational toggles (physical shop or online workflow processing).
- Mock payment integration simulating transaction success without external financial APIs.
- Post-service user review and rating feedback module.

Out-of-Scope Features (Limitations)

- AI-based design recommendations or automatic measurement estimations.
- Advanced business intelligence telemetry, predictive analytics, or dashboard insights.
- Live peer-to-peer instant messaging chat window interfaces (basic notifications only).
- Interactive logistics package delivery tracking with live maps integrations.
- Production-ready secure real payment gateway implementation (mock transactions only).
- Native mobile application software development (web-only responsive MVP framework).

Engineering Framework Specifications

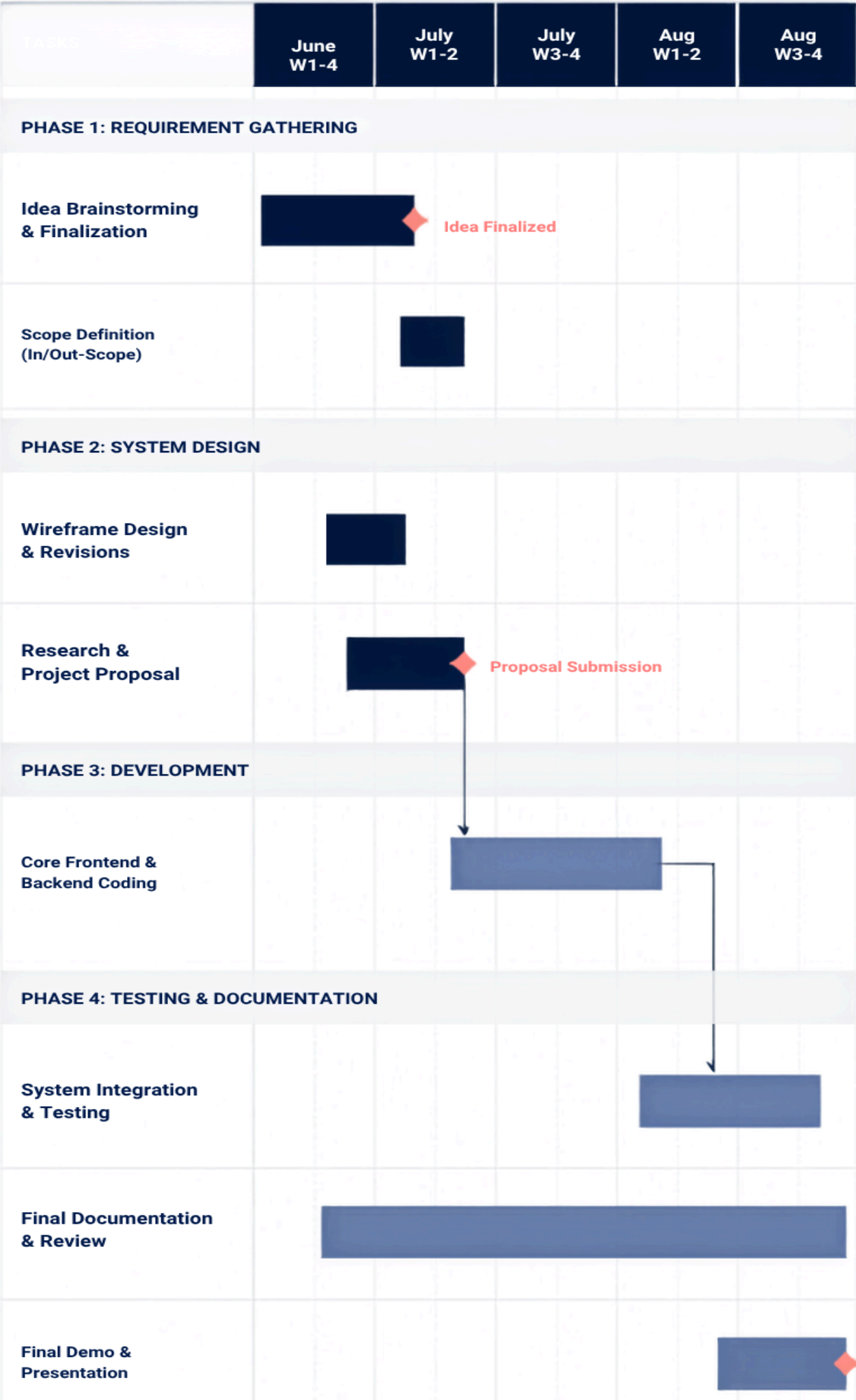
- **Functional Requirements:** Users must be able to cleanly register, login, and initialize roles. System tracking variables must dynamically match and deliver real-time data to dashboard components.
- **Non-Functional Requirements:** Simple UI/UX usability matching low operational friction, fast server response latencies (<3 seconds), and structural transactional database reliability.

3 Initial Timeline (Gantt Chart)

3.1 Milestone Schedule and Status Matrix

The project timeline chronicles historical task parameters from ideation through upcoming completion markers over the 2-month lifecycle.

 Completed	Planned / In Progress	Milestone	Dependency
---	-----------------------	-----------	------------



3.2 Risk Mitigation & Contingency Strategy

- **Identification:** Technical bottlenecks include scope creep regarding real-time notification components and multimedia storage constraints (large images and voice file inputs) inside the restricted 2-month execution matrix.
- **Contingency Planning:** Audio playback structures can utilize lightweight direct URL retrieval variables, and push mechanics will be down-scaled to standard dashboard flag mutations if delivery boundaries are threatened.

4. Team Roles & Responsibilities

4.1 Group Personnel Allocations

Full Name	Student ID	Assigned Role(s)	Specific Responsibilities
Oshanda Induwara	CIT-24-02-0125	Backend Developer / Full Stack / QA	API development using FastAPI, database schema integration, and backend functional testing.
Fathima Zainab Ilyas	CIT-24-02-0182	Project Manager / Business Analyst / Backend Developer / QA	Requirement analysis, project documentation, sprint coordination, contributing to backend development, and overall quality assurance.
Lamindu Rathnayake	CIT-24-02-0030	Full Stack Developer / QA	Supporting both frontend and backend development, feature integration, and component

			testing.
Sakila Induwara	CIT-24-02-0113	UI/UX Engineer / QA	Designing wireframes and interactive prototypes in Figma, ensuring user experience consistency, and UI testing.
Ahmed Rafnas	CIT-24-02-0064	Frontend Developer / QA	Implementing the user interface using React, ensuring mobile responsiveness, frontend logic, and client-side testing.

4.2 Equipment, Infrastructure & Resources

- Frontend Engine Architecture: React Web Framework
- Backend Engine Architecture: Python FastAPI
- Database Infrastructure: Supabase Platform (PostgreSQL System Layer)
- Identity Service Handler: Supabase Auth Framework
- Cloud Staging Environment: Google Cloud Platform (GCP)
- Media Assets Management: Cloudinary CDN Integration
- UX Prototyping Suite: Figma Collaborative Editor
- Agile Management Tool: Jira Software Sprint Dashboards

4.3 Financial Feasibility & Budget Summary

Personnel & Resource Costs: LKR 0.00 (Academic configuration designed by group students; utilizes open-source components and free cloud tier quotas exclusively). Financial variances are completely mitigated by feature scaling variables rather than budget shifts.